# CSA17 – ARTIFICIAL INTELLIGENCE LAB

# Experiments 5 to 10

**Name: D.Vishnu Vardhan Reddy**

**Register Number: 192325039**

## Experiment 5: Missionaries and Cannibals Problem

### Aim

To move three missionaries and three cannibals across a river using a two-person boat without ever allowing cannibals to outnumber missionaries on either bank where missionaries are present.

### Algorithm

1. Represent a state as (missionaries_left, cannibals_left, boat_side).
2. Generate boat trips carrying one or two people.
3. Reject states where counts are outside 0–3 or cannibals outnumber missionaries on a bank with missionaries.
4. Use BFS to find a path from (3, 3, left) to (0, 0, right).
5. Print each state in the solution path.

### Program (Python)

```
from collections import deque

start, goal = (3, 3, 0), (0, 0, 1)  # 0=left, 1=right
moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

def valid(m, c):
    mr, cr = 3-m, 3-c
    return (0 <= m <= 3 and 0 <= c <= 3 and
            (m == 0 or m >= c) and (mr == 0 or mr >= cr))

q = deque([start])
parent = {start: None}
while q:
    s = q.popleft()
    if s == goal:
        break
    m, c, side = s
    for dm, dc in moves:
        nm, nc = (m-dm, c-dc) if side == 0 else (m+dm, c+dc)
        ns = (nm, nc, 1-side)
        if valid(nm, nc) and ns not in parent:
            parent[ns] = s
            q.append(ns)

if goal in parent:
    path, cur = [], goal
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    for m, c, side in reversed(path):
        print(f"Left: M={m}, C={c}; Right: M={3-m}, C={3-c}; Boat:",
              "Right" if side else "Left")
else:
    print("No solution found")
```

### Input

```
Missionaries = 3, Cannibals = 3, Boat capacity = 2
```

### Output

```
The program prints a valid sequence of states from (3, 3, Left) to (0, 0, Right), ensuring missionaries are never outnumbere
```

# Experiment 6: Vacuum Cleaner Problem

## Aim

To model a two-room vacuum cleaner world and clean both rooms using simple condition-action rules.

## Algorithm

1. Represent each room as Clean or Dirty and record the vacuum's location.
2. If the current room is dirty, clean it.
3. Otherwise move to the other room if it is not already there.
4. Repeat until both rooms are clean.
5. Display each action and the final state.

## Program (Python)

```
rooms = {"A": "Dirty", "B": "Dirty"}
location = "A"

while "Dirty" in rooms.values():
    print("Location:", location, "| State:", rooms)
    if rooms[location] == "Dirty":
        rooms[location] = "Clean"
        print("Action: Suck")
    else:
        location = "B" if location == "A" else "A"
        print("Action: Move to", location)

print("Final state:", rooms)
print("Both rooms are clean.")
```

## Input

```
Room A = Dirty, Room B = Dirty, initial location = A
```

## Output

```
Location: A | State: {'A': 'Dirty', 'B': 'Dirty'}
Action: Suck
Location: A | State: {'A': 'Clean', 'B': 'Dirty'}
Action: Move to B
Location: B | State: {'A': 'Clean', 'B': 'Dirty'}
Action: Suck
Final state: {'A': 'Clean', 'B': 'Clean'}
Both rooms are clean.
```

# Experiment 7: Implement Breadth-First Search (BFS)

## Aim

To traverse a graph using Breadth-First Search, visiting nodes level by level.

## Algorithm

1. Put the start node in a queue and mark it visited.
2. Remove a node from the front of the queue and print it.
3. Add each unvisited adjacent node to the queue and mark it visited.
4. Continue until the queue is empty.

## Program (Python)

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [], "E": [], "F": []
}

def bfs(start):
    visited = {start}
    q = deque([start])
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                q.append(neighbor)
    return order

print("BFS traversal:", bfs("A"))
```

## Input

```
Start node: A
Graph: A-B, A-C, B-D, B-E, C-F
```

## Output

```
BFS traversal: ['A', 'B', 'C', 'D', 'E', 'F']
```

# Experiment 8: Implement Depth-First Search (DFS)

## Aim

To traverse a graph using Depth-First Search by exploring each branch as deeply as possible before backtracking.

## Algorithm

1. Start at the given node and mark it visited.
2. Print the current node.
3. Recursively visit each adjacent node that has not been visited.
4. Stop when all reachable nodes have been visited.

## Program (Python)

```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [], "E": [], "F": []
}

def dfs(node, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    visited.add(node)
    order.append(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(neighbor, visited, order)
    return order

print("DFS traversal:", dfs("A"))
```

## Input

```
Start node: A
Graph: A-B, A-C, B-D, B-E, C-F
```

## Output

```
DFS traversal: ['A', 'B', 'D', 'E', 'C', 'F']
```

# Experiment 9: Travelling Salesman Problem

## Aim

To find the shortest tour that visits every city exactly once and returns to the starting city using exhaustive search.

## Algorithm

1. Fix city A as the starting city.
2. Generate every permutation of the remaining cities.
3. Calculate each tour's total distance, including the return to A.
4. Keep the tour with the smallest distance.
5. Print the best tour and its distance.

## Program (Python)

```
from itertools import permutations

cities = ["A", "B", "C", "D"]
dist = {
    ("A","B"):10, ("A","C"):15, ("A","D"):20,
    ("B","C"):35, ("B","D"):25, ("C","D"):30
}

def distance(a, b):
    return dist.get((a,b), dist.get((b,a)))

best_tour, best_cost = None, float("inf")
for order in permutations(cities[1:]):
    tour = ("A",) + order + ("A",)
    cost = sum(distance(tour[i], tour[i+1])
               for i in range(len(tour)-1))
    if cost < best_cost:
        best_tour, best_cost = tour, cost

print("Best tour:", " -> ".join(best_tour))
print("Minimum cost:", best_cost)
```

## Input

```
Cities: A, B, C, D
Distances: AB=10, AC=15, AD=20, BC=35, BD=25, CD=30
```

## Output

```
Best tour: A -> B -> D -> C -> A
Minimum cost: 80
```

# Experiment 10: Implement A* Search Algorithm

## Aim

To find a minimum-cost path from a start node to a goal node using A* search with path cost g(n) and heuristic h(n).

## Algorithm

1. Put the start node in a priority queue with f(n)=g(n)+h(n).
2. Remove the node with the smallest f-value.
3. If it is the goal, reconstruct and print the path.
4. For each neighbor, update its path cost and parent if a cheaper path is found.
5. Add updated nodes to the priority queue and continue.

## Program (Python)

```
import heapq

graph = {
    "A": {"B": 1, "C": 4},
    "B": {"C": 2, "D": 5},
    "C": {"D": 1},
    "D": {}
}
heuristic = {"A": 4, "B": 3, "C": 1, "D": 0}

def astar(start, goal):
    pq = [(heuristic[start], 0, start)]
    parent = {start: None}
    cost = {start: 0}
    while pq:
        f, g, node = heapq.heappop(pq)
        if node == goal:
            path, cur = [], goal
            while cur is not None:
                path.append(cur)
                cur = parent[cur]
            return path[::-1], g
        if g != cost[node]:
            continue
        for nxt, weight in graph[node].items():
            new_cost = g + weight
            if new_cost < cost.get(nxt, float("inf")):
                cost[nxt] = new_cost
                parent[nxt] = node
                heapq.heappush(pq, (new_cost + heuristic[nxt],
                                    new_cost, nxt))
    return None, float("inf")

path, cost = astar("A", "D")
print("Path:", " -> ".join(path))
print("Total cost:", cost)
```

## Input

```
Start: A
Goal: D
Graph edges: A-B=1, A-C=4, B-C=2, B-D=5, C-D=1
```

## Output

```
Path: A -> B -> C -> D
Total cost: 4
```